```
print("Computer Programming 2026\n")
print("Department of Geography\n")
print("9 April 2026")

print("Week 7")
```

TUPLES

Tuples

A tuple is a collection that is ordered and **unchangeable**

```
mytuple = ("apple", "banana", "cherry")
```

We cannot change, add, or remove items after the tuple has been created

Tuples

- Create a tuple:

```
mytuple = ("apple", "banana", "cherry")
mytuple = tuple(("apple", "banana", "cherry"))
```

- Tuples allow duplicate values:

```
mytuple = ("apple", "banana", "cherry", "apple", "cherry")
```

- Print the number of items in the tuple:

```
print(len(mytuple))
```

- Create tuple with one item:

~~`mytuple = ("apple")`~~ **WRONG**

`mytuple = ("apple",)` **CORRECT**

Tuples

- Tuple of strings: `tuple1 = ("apple", "banana", "cherry")`
- Tuple of integers: `tuple2 = (1, 5, 7, 9, 3)`
- Tuple of boolean values: `tuple3 = (True, False, False)`

- Tuple with strings, integers, and boolean values:

```
tuple4 = ("abc", 34, True, 40, "male")
```

Tuples

- Print the second item of the tuple:

```
mytuple = ("apple", "banana", "cherry")
print(mytuple[1])
```

- Print the last item of the tuple:

```
mytuple = ("apple", "banana", "cherry")
print(mytuple[-1])
```

- Print the third, fourth and fifth item of the tuple:

```
mytuple=("apple","banana","cherry","orange","kiwi","melon","mango")
print(mytuple[2:5])
```

Tuples

- Print the first four items of the tuple:

```
mytuple=("apple","banana","cherry","orange","kiwi","melon","mango")
print(mytuple[:4])
```

- Print the last five items of the tuple:

```
mytuple=("apple","banana","cherry","orange","kiwi","melon","mango")
print(mytuple[2:])
```

- Print the fourth, fifth and sixth item in the tuple:

```
mytuple=("apple","banana","cherry","orange","kiwi","melon","mango")
print(mytuple[-4:-1])
```

Tuples

- Check if “apple” is present in the tuple:

```
mytuple = (“apple”, “banana”, “cherry”)
if “apple” in mytuple:
    print(“Yes, ‘apple’ is in the fruits tuple”)
```

- Change the second item of the tuple:

```
mytuple = (“apple”, “banana”, “cherry”)
y = list(mytuple)
y[1] = “kiwi”
mytuple = tuple(y)
```

Tuples

- Insert an item at the end:

```
mytuple = (“apple”, “banana”, “cherry”)
y = list(mytuple)
y.append(“orange”)
mytuple = tuple(y)

mytuple = (“apple”, “banana”, “cherry”)
y = (“orange”,)
mytuple += y
```

Tuples

- Join two tuples:

```
tuple1 = (“a”, “b”, “c”)
tuple2 = (1, 2, 3)
tuple3 = tuple1 + tuple2
```

- Double the content of a tuple

```
fruits = (“apple”, “banana”, “cherry”)
mytuple = fruits * 2
```

Tuples

- Remove “apple”:

```
mytuple = (“apple”, “banana”, “cherry”)
y = list(mytuple)
y.remove(“apple”)
mytuple = tuple(y)
```

- Delete the entire tuple:

```
mytuple = (“apple”, “banana”, “cherry”)
del mytuple
```

Tuples

- Unpacking a tuple:

```
fruits = (“apple”, “banana”, “cherry”)
(green, yellow, red) = fruits
```

- Unpacking a tuple:

```
fruits=(“apple”,“banana”,“cherry”,“strawberry”,“raspberry”)
(green, yellow, *red) = fruits
```

- Unpack the tuple by saving the tropical fruits in a list:

```
fruits=(“apple”,“mango”,“papaya”,“pineapple”,“cherry”)
(green, *tropic, red) = fruits
```

SETS

Sets

A set is a collection that is *unordered* and *unchangeable*

```
myset = {"apple", "banana", "cherry"}
```

Once a set is created, we cannot change its items, but we can remove items and add new items

The items in a set do not have a defined order, and cannot be referred to by index

Sets

- Create a set:

```
myset = {"apple", "banana", "cherry"}
myset = set(("apple", "banana", "cherry"))
```

- Sets do not allow duplicate values:

```
myset = {"apple", "banana", "cherry", "apple"}
```

Duplicate values will be ignored!

```
myset = {"apple", "banana", "cherry", True, 1, 2}
```

True and 1 are considered the same value!

```
myset = {"apple", "banana", "cherry", False, True, 0}
```

False and 0 are considered the same value!

Sets

- Print the number of items in the set: `print(len(myset))`

- Set of strings: `set1 = {"apple", "banana", "cherry"}`
- Set of integers: `set2 = {1, 5, 7, 9, 3}`
- Set of boolean values: `set3 = {True, False, False}`

- Set with strings, integers, and boolean values:

```
set4 = {"abc", 34, True, 40, "male"}
```

Sets

- Check if “banana” is present in the set:

```
myset = {“apple”, “banana”, “cherry”}
if “banana” in myset:
    print(“Yes, ‘banana’ is in the fruits set”)
```

- Change the second item of the set:

```
myset = {“apple”, “banana”, “cherry”}
```

Once a set is created, you cannot change its items

Transform it into a list (???)

- Loop in a set:

```
for x in myset:
    print(x)
```

Sets

- Add an item to a set:

```
myset = {“apple”, “banana”, “cherry”}
myset.add(“orange”)
```

- Add items to a set from another set:

```
myset = {“apple”, “banana”, “cherry”}
tropical = {“pineapple”, “mango”, “papaya”}
myset.update(tropical)
```

- Add items to a set from a list:

```
myset = {“apple”, “banana”, “cherry”}
mylist = [“kiwi”, “orange”]
myset.update(mylist)
```

Sets

- Remove “banana”:

```
myset = {“apple”, “banana”, “cherry”}
myset.remove(“banana”)
```
If the item does not exist, it raises an error

```
myset.discard(“banana”)
```
If the item does not exist, it does not raise an error

```
myset.pop(1)
```
WRONG! It removes a random item!

- Clear the set content:

```
myset = {“apple”, “banana”, “cherry”}
myset.clear()
```

- Delete the entire set:

```
myset = {“apple”, “banana”, “cherry”}
del myset
```

Sets

- Join two sets:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)          set3 = set1 | set2

set1.update(set2)
```

Any duplicate item will be excluded

- Keep only the duplicates

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.intersection(y)            z = x & y

x.intersection_update(y)
```

Sets

- Keep the items that are not present in both sets:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.symmetric_difference(y)

x.symmetric_difference_update(y)
```

- Keep the items that are not present in both sets:

```
x = {"apple", "banana", "cherry", True}
y = {"google", 1, "apple", 2}
z = x.symmetric_difference(y)
```

True and 1 are considered the same value!

Treated as duplicates!

DICTIONARIES

Dictionaries

A dictionary is a collection that is *ordered* and *changeable*

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

Dictionaries store data values in ***key:value*** pairs

(Dictionaries are ordered from Python 3.7)

Dictionaries

- Create a dictionary:

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

```
mydict = dict(name = "John", age = 36, country = "Norway")
```

- Dictionaries do not allow duplicate keys:

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
    "year": 2020
}
```

Duplicate keys will overwrite existing values!

Dictionaries

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

- Print the brand value of the dictionary:

```
print(mydict["brand"])
```

- Print the number of items in the dictionary: `print(len(mydict))`

- The values in dictionary items can be of any data types:

```
mydict = {
    "brand": "Ford",
    "electric": False,
    "year": 1964,
    "colors": ["red", "white", "blue"]
}
```

Dictionaries

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

- Get the value of the "model" key:

```
x = mydict["model"]
```

```
x = mydict.get("model")
```

```
x = mydict.get("aaa", "This does not exist")
```

- Get a list of the keys in the dictionary:

```
x = list(mydict.keys())
```

- Get a list of the values in the dictionary:

```
x = list(mydict.values())
```

Dictionaries

```
mydict = {
     "brand": "Ford",
     "model": "Mustang",
     "year": 1964
}
```

- Get a list of the key:value pairs:

```
x = list(mydict.items())
```

List of tuples!

- Check if "model" is present in the dictionary:

```
if "model" in mydict:
    print("Yes, 'model' is one of the keys in the dictionary")
```

Dictionaries

```
mydict = {
     “brand”: “Ford”,
     “model”: “Mustang”,
     “year”: 1964
}
```

- Change the year to 2018:

```
mydict[“year”] = 2018
mydict.update({“year”: 2018})
```

- Add an item to the dictionary:

```
mydict[“color”] = “red”
mydict.update({“color”: “red”})
```

Dictionaries

- Remove an item:

```
mydict.pop("model")
```

```
del mydict["model"]
```

- Remove the last inserted item:

```
mydict.popitem()
```

- Delete the entire dictionary:

```
del mydict
```

- Clear the dictionary content

```
mydict.clear()
```

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

Dictionaries

- Iterate over the key names of the dictionary:

```
for x in mydict.keys():
    print(x)

for x in mydict:
    print(x)
```

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

- Iterate over the values of the dictionary:

```
for x in mydict.values():
    print(x)
```

- Iterate over both keys and values:

```
for x, y in mydict.items():
    print(x, y)
```

Dictionaries

```
mydict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

- Copy a dictionary:

~~`dict2 = mydict`~~ **WRONG**

`newdict = mydict.copy()` **CORRECT**

`newdict = dict(mydict)` **CORRECT**

Dictionaries

- A dictionary can contain dictionaries (**nested dictionaries**):

```
family = {
    "child1" : {
        "name" : "Emil",
        "year" : 2004
    },
    "child2" : {
        "name" : "Tobias",
        "year" : 2007
    }
    "child3" : {
        "name" : "Linus",
        "year" : 2011
    }
}
```

Dictionaries

- You can also add some dictionaries into a new dictionary:

```
family = {
    "child1" : child1,
    "child2" : child2,
    "child3" : child3
}
```

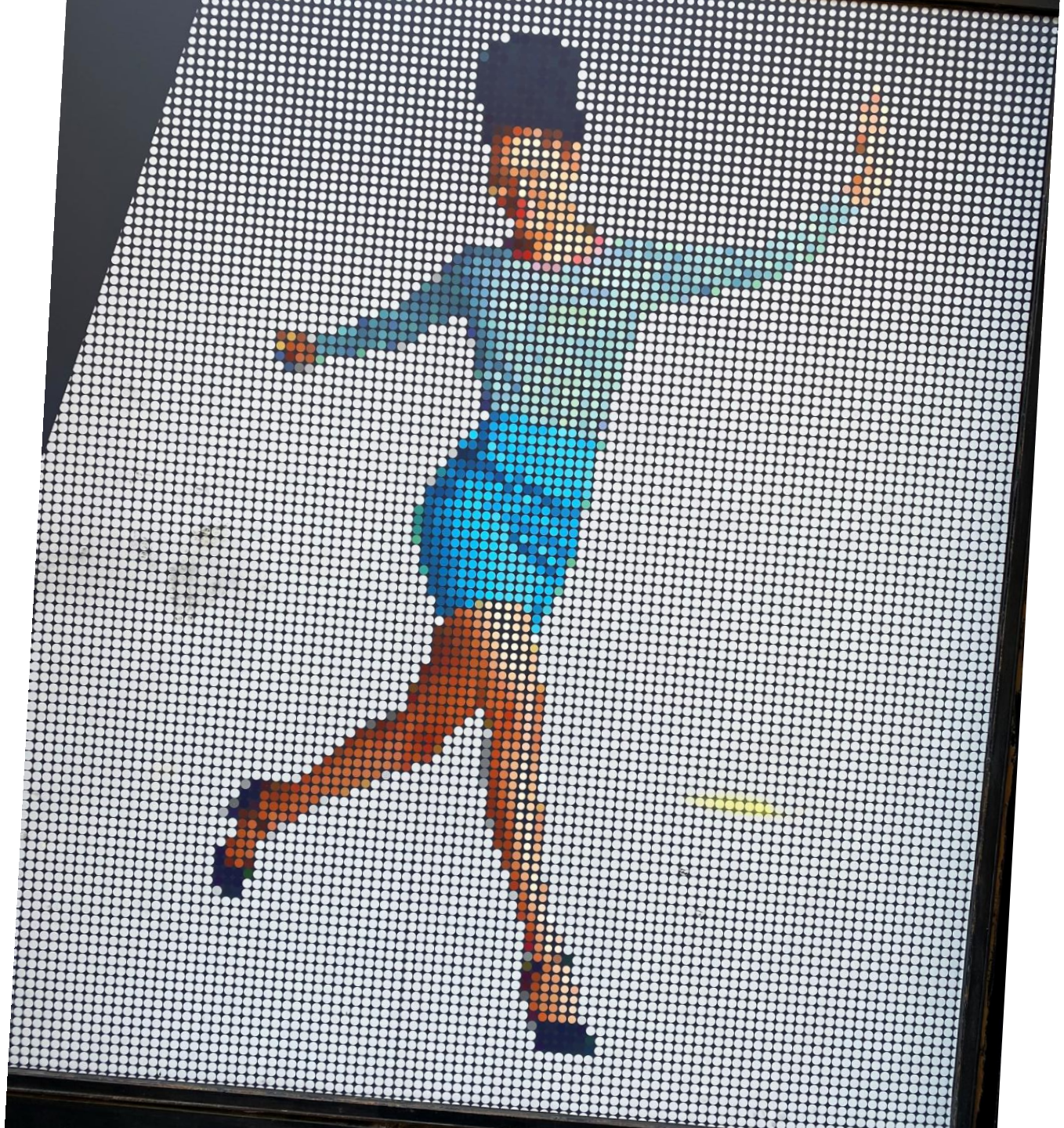
- Print the name of child 2:

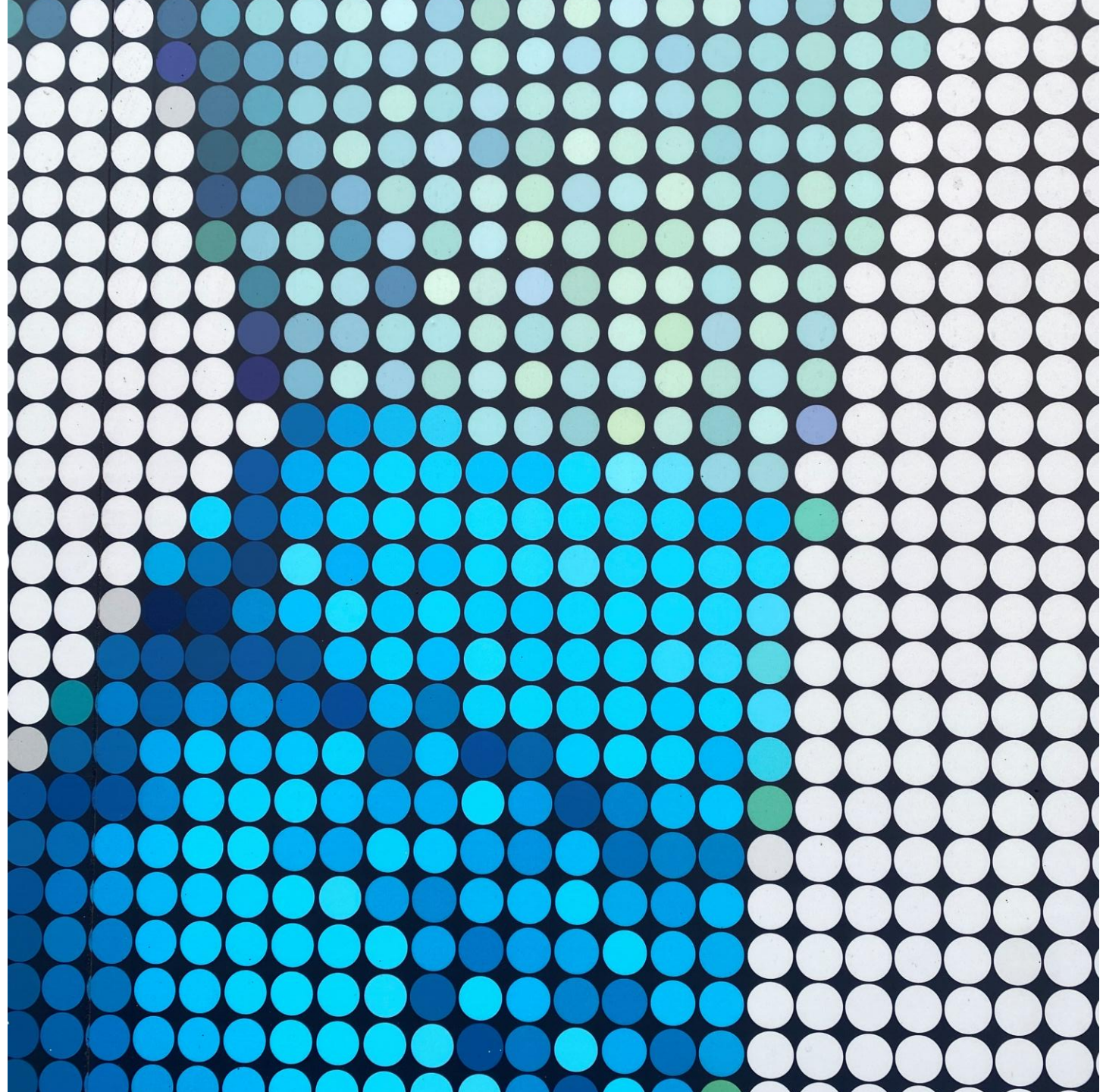
```
print(family["child2"]["name"])
```

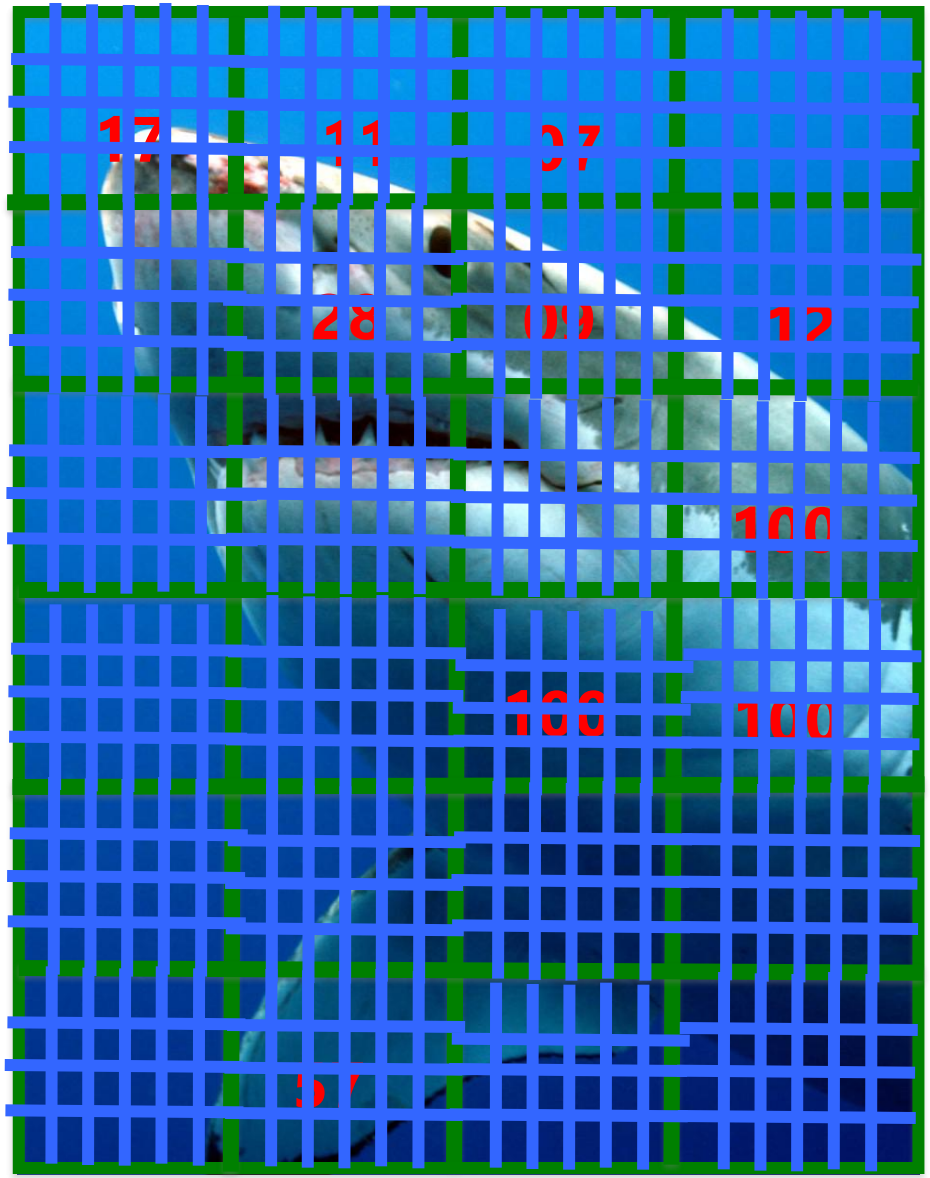
```
child1 = {
    "name" : "Emil",
    "year" : 2004
}

child2 = {
    "name" : "Tobias",
    "year" : 2007
}

child3 = {
    "name" : "Linus",
    "year" : 2011
}
```







How do we represent it?

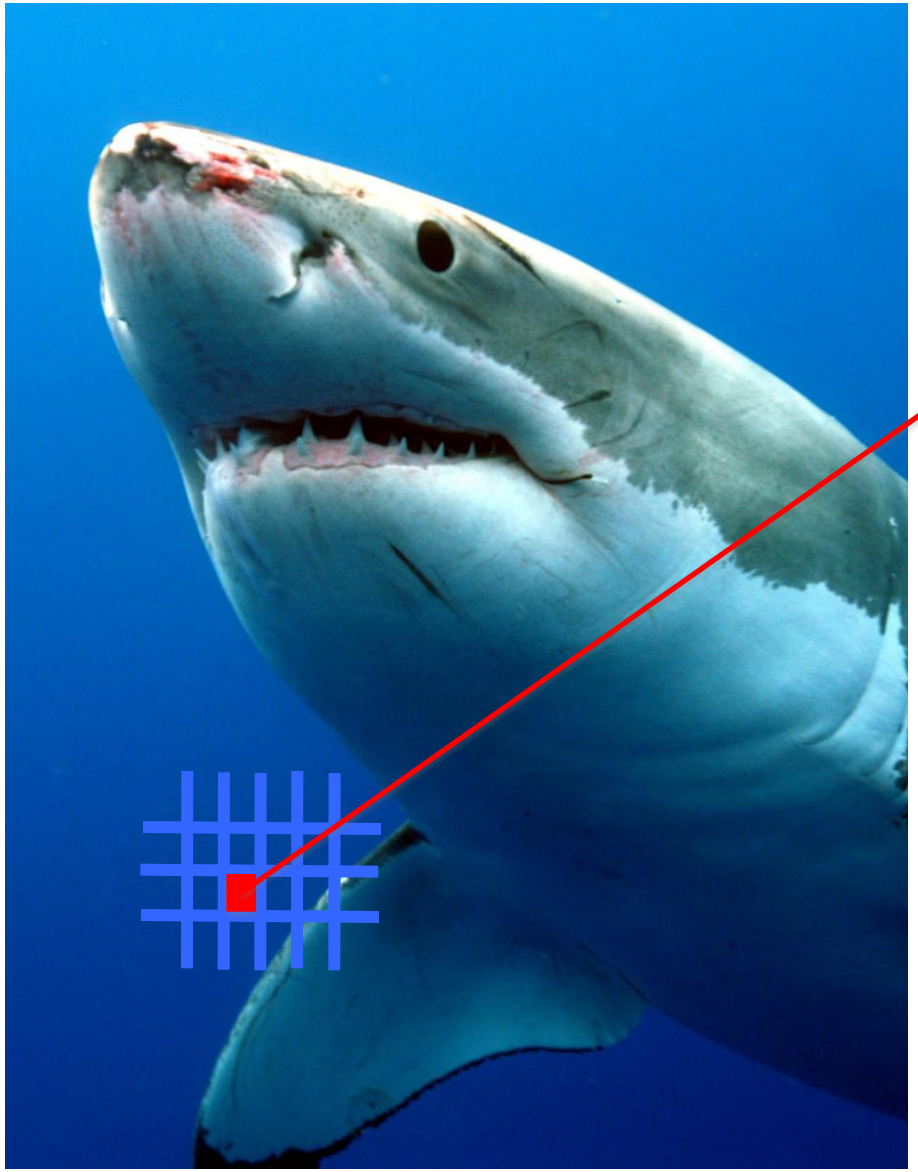
We create a grid
(aka matrix)

But what does the matrix contain?

The *points*
(observations)

How do I represent the *points*?

How do I increase the *resolution*?



The pixel *is the color*

How do I obtain the blue?

With the format RGB

R: 33
G: 91
B: 191

PIXEL = R, G, B

ARRAYS

Arrays

An image is represented as a 3D array:

- the first dimension's size is the height,
- the second is the width,
- the third is the number of color channels, in this case: red, green and blue (RGB).

In other words, for each pixel there is a 3D vector containing the intensities of red, green and blue, each between 0.0 and 1.0 (or between 0 and 255)

Some images may have less channels, such as gray-scale images (one channel), or more channels, such as images with an additional alpha channel for transparency, or satellite images which often contain channels for many light frequencies (e.g., infrared).

Arrays

- 1D arrays are called vectors, 2D arrays are called matrices, 3D arrays are called 3D matrices (or simply 3D arrays), and so on…
- Arrays can assume any chosen number of dimensions
- How to define an array in Python?

Numpy library

```
import numpy as np
```

Matrices

- In mathematics, a **matrix** is a rectangular representation of objects (usually numbers), organized in **rows** (horizontal) and **columns** (vertical)
- In general, a matrix has *m* rows and *n* columns, with *m* and *n* being positive integers. Often, a matrix is described indicating with a_{ij} the generic **element** located at the *i*-th row and *j*-th column
- Vectors can be considered as matrices with only one row or only one column (called **row vector** and **column vector**)

$$\begin{bmatrix} 1 & 2 & 3 \\ 1 & -2 & 0 \\ 4.5 & 0 & 2 \\ 6 & 1 & 5 \end{bmatrix} \qquad \begin{bmatrix} 3 & \frac{7}{2} & -9 \end{bmatrix} \qquad \begin{bmatrix} 7 \\ 0 \\ \pi \end{bmatrix}$$

Matrices: *the elements*

- The notation $A = (a_{i,j})$ indicates that A is a matrix and its elements are denoted as $a_{i,j}$

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$$

- The elements having the same indices of row and column (the elements in the form a_{ii}) represent the **main diagonal** of the matrix.

- If the matrix is called A, the element $a_{i,j}$ can be indicated also as $A[i,j]$

Numpy Arrays

```
import numpy as np
```

Initializing a Numpy Array:

```
np.empty(shape, dtype=float)
                        np.empty((2, 2), dtype=int)

np.zeros(shape, dtype=float)
np.ones(shape, dtype=float)
                                np.random.rand(2, 2)
                  np.random.randint(1, 100, (2, 2))
```

Numpy Arrays

Examples

```
>>> np.zeros(5)
array([ 0.,  0.,  0.,  0.,  0.])
```

```
>>> np.zeros((5,), dtype=int)
array([0, 0, 0, 0, 0])
```

```
>>> np.zeros((2, 1))
array([[ 0.],
       [ 0.]])
```

```
>>> s = (2,2)
>>> np.zeros(s)
array([[ 0.,  0.],
       [ 0.,  0.]])
```

Examples

```
>>> np.ones(5)
array([1., 1., 1., 1., 1.])
```

```
>>> np.ones((5,), dtype=int)
array([1, 1, 1, 1, 1])
```

```
>>> np.ones((2, 1))
array([[1.],
       [1.]])
```

```
>>> s = (2,2)
>>> np.ones(s)
array([[1.,  1.],
       [1.,  1.]])
```

matrix of 20×30 integer elements (600 variables):

```
A = np.zeros((20, 30), int)
```

3D matrix of $20 \times 20 \times 30$ float elements (12000 variables):

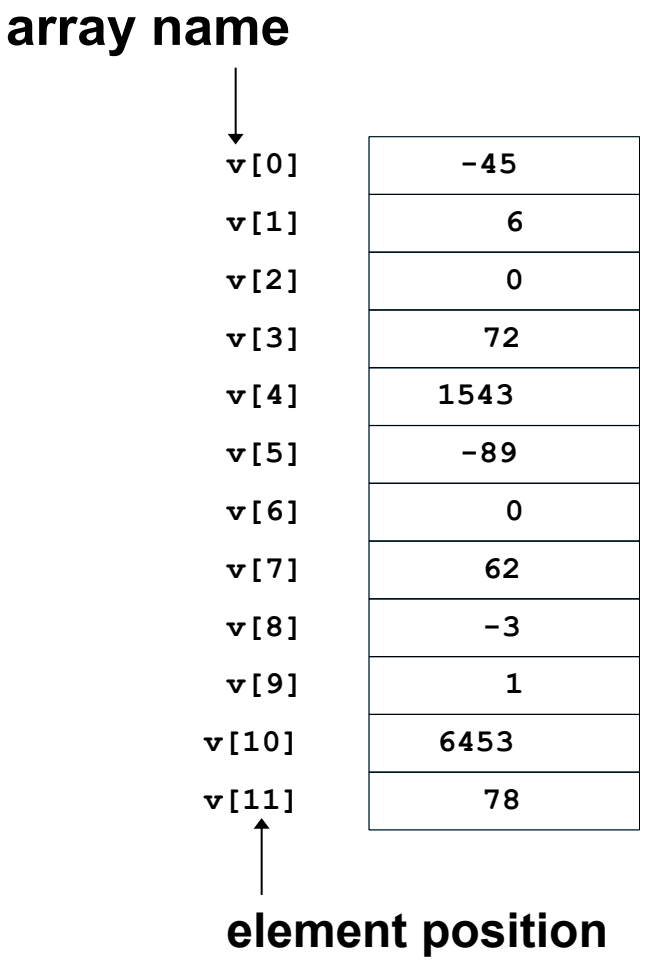
```
F = np.zeros((20, 20, 30))
```

Array

- Group of consecutive cells
  - Representation of variable groups
    - With the same name AND **THE SAME TYPE**

- To refer to an ***element***, we must specify:
  - The name of the array
  - The ***position*** of the element (index)

- Syntax: *array_name*[*element_position*]
  - The first element has index **0**
  - The n-th element of the array **`v`** is **`v[n-1]`**

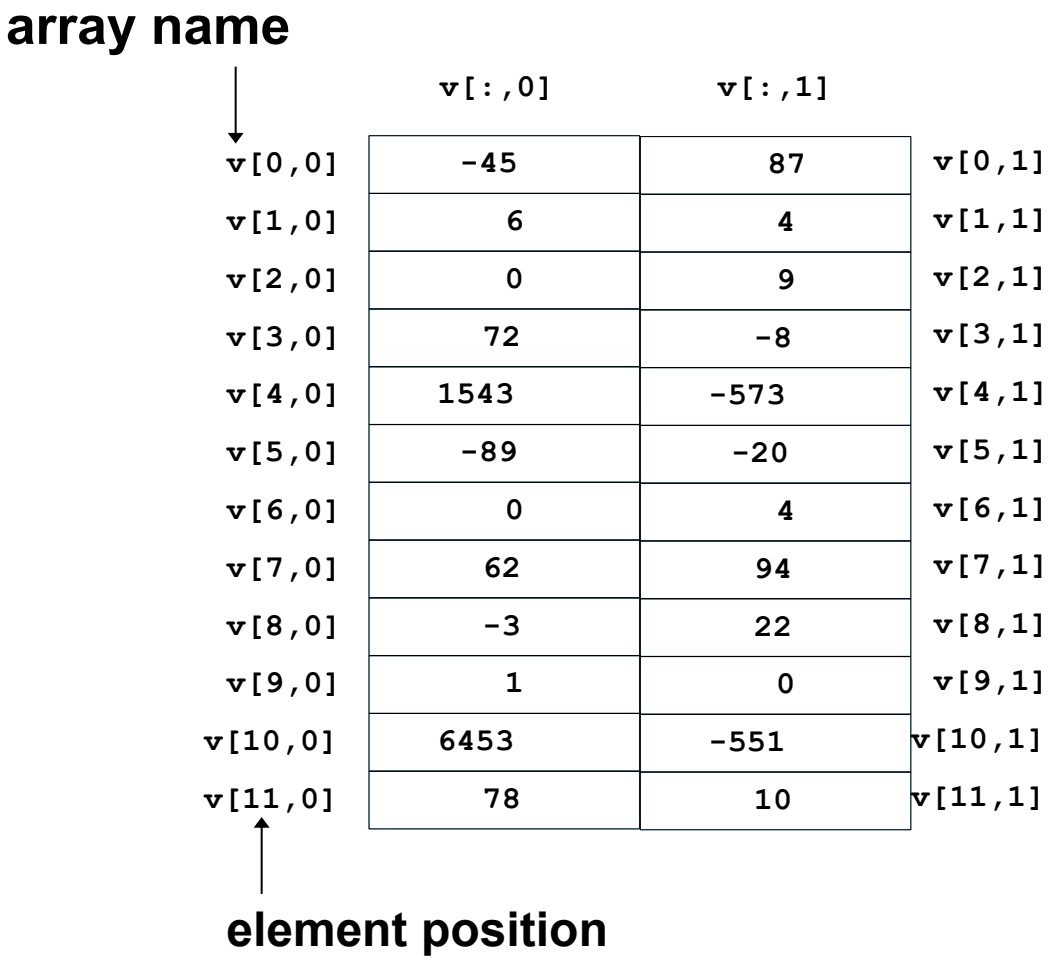


Array

- Group of consecutive cells
  - Representation of variable groups
    - With the same name AND **THE SAME TYPE**

- To refer to an ***element***, we must specify:
  - The name of the array
  - The ***position*** of the element (index)

- Syntax: *array_name*[*element_position*]
  - The first element has index **0**
  - The n-th element of the array **`v`** is **`v[n-1]`**



Array

- The elements of an array are simple values:

```
myarray[0,3,1,9] = 3
print(myarray[0,3,1,9])
> 3
myarray[0,0,1] = input("...: ")
> 17
```

`myarray[0,0,1]` assumes value 17

- You can also use **expressions** as indices:

`myarray[5-2,7+4]` is equivalent to `myarray[3,11]`

if `x = 3` and `y = 5`, then `myarray[x,y]` is equivalent to `myarray[3,5]`

Exercise

- Given 10 integers inserted by the user, display such 10 integers in the **reversed order** of insertion (use array)

```
DIM = 10
data = np.zeros(DIM, int)

for i in range(DIM):
    data[i] = int(input(f"Insert the number (choice {i+1}): "))

print("\n")
for i in range(DIM):
    print(f"The number (choice {i+1}) is: {data[i]}")

print("\n")
for i in range(DIM,0,-1):
    print(f"The number (choice {i}) is: {data[i-1]}")
```

Exercise

- Given 10 integers inserted by the user, display such 10 integers in the **reversed order** of insertion (use array)

```
DIM = 10
data = np.zeros(DIM, int)

for i in range(DIM):
    data[i] = int(input(f"Insert the number (choice {i+1}): "))
```

```
data = []
DIM = 10

for i in range(DIM):
    data.append(int(input(f"Insert the number (choice {i+1}): ")))
```

A short summary…

- Array: collection of homogeneous data
  - Declare the type of its elements
  - Declare its dimension

```
data = np.zeros(DIM, int)
```

- How to access data in an array?
  - Example of *writing*

```
for i in range(DIM):
    data[i] = int(input(f"Insert the number (choice {i+1}): "))
```

  - Example of *reading*

```
for i in range(DIM):
    print(f"The number (choice {i+1}) is: {data[i]}")
```

Matrices: sum

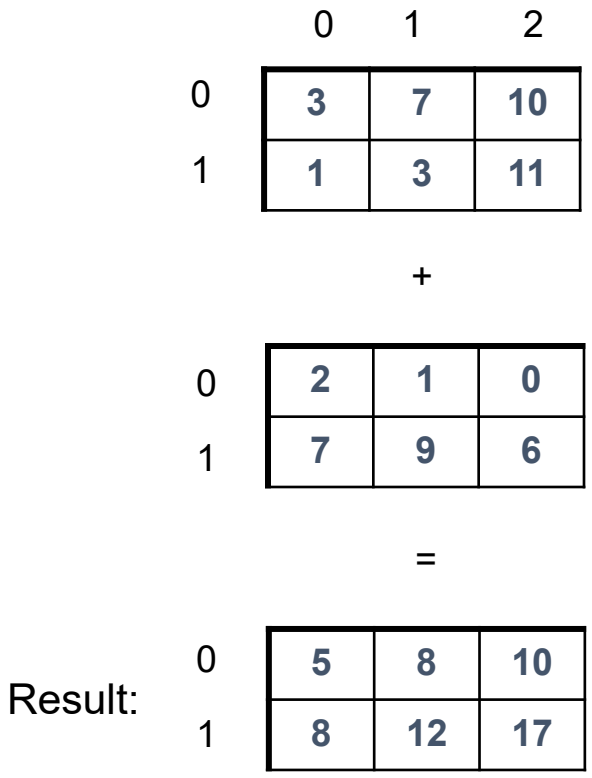
- Two matrices *A* and *B* can be summed up if they have the same dimensions
- Their sum *A* + *B* is defined as the matrix whose elements are obtained summing up the corresponding elements of *A* and *B*.
- Formally: $(A + B)_{i,j} := A_{i,j} + B_{i,j}$

$$\begin{bmatrix} 1 & 3 & 2 \\ 1 & 0 & 0 \\ 1 & 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 5 \\ 7 & 5 & 0 \\ 2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1+0 & 3+0 & 2+5 \\ 1+7 & 0+5 & 0+0 \\ 1+2 & 2+1 & 2+1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 7 \\ 8 & 5 & 0 \\ 3 & 3 & 3 \end{bmatrix}.$$

A + B

Exercise

- Ask the user to insert the following matrices, and print out on screen the result of their sum.




```
dim1 = 2
dim2 = 3
a = np.zeros((dim1, dim2), int)
b = np.zeros((dim1, dim2), int)

for i in range(dim1):
    for j in range(dim2):
        a[i,j] = int(input("Insert a value for matrix a: "))

for i in range(dim1):
    for j in range(dim2):
        b[i,j] = int(input("Insert a value for matrix b: "))

print("\nThe sum is:\n", a + b)
```

Exercise

- Ask the user to insert the following matrices, and print out on screen the result of their sum.

	0	1	2
0	3	7	10
1	1	3	11

+

	0	1	2
0	2	1	0
1	7	9	6

=

Result:

	0	1	2
0	5	8	10
1	8	12	17

```
dim1 = 2
dim2 = 3
a = np.zeros((dim1, dim2), int)
b = np.zeros((dim1, dim2), int)

for i in range(dim1):
    for j in range(dim2):
        a[i,j] = int(input("Insert a value for matrix a: "))

for i in range(dim1):
    for j in range(dim2):
        b[i,j] = int(input("Insert a value for matrix b: "))

print("\nThe sum is:\n")
c = a + b
for i in range(dim1):
    for j in range(dim2):
        print(c[i,j], end="\t")
    print("")
```

See you
next week!